

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Multidisciplinary Project - Artificial Intelligence Oriented

Course Project

“Motion Capture System”

Instructor(s): Nguyễn Tuấn Khôi
Ngô Hoàng Anh

Students: Lưu Tôn Thịnh	2353139
Huỳnh Văn Thanh	2353083
Trần Thái Tài	2353065
Ngô Vĩnh Quang	2352968
Nguyễn Bùi Huỳnh Khương	2352637

HO CHI MINH CITY, June 2026



Acknowledgments

We would like to express our sincere gratitude to our teachers Nguyễn Tuấn Khôi and Ngô Hoàng Anh. They directly guided us, provided materials, and supported us throughout this project. Their instructions, guidance, and professional knowledge helped us complete this report with a deeper understanding of the subject.

We greatly appreciate the contributions of our teachers and hope to continue receiving their valuable guidance in future projects.

We wish our teachers good health, happiness, and even greater success in their teaching and research careers.

Member list & Workload

No.	Fullname	Student ID	Problems	% done
1	Lưu Tôn Thịnh	2353139	- Code camera calibration and synchronization - Setup YOLOv26, MediaPipe, and OpenPose	100%
2	Huỳnh Văn Thanh	2353083	- Collect metric data and format tables - Write Results, Discussion, and Conclusion	100%
3	Trần Thái Tài	2353065	- Code 3D triangulation and RANSAC - Code post-processing and AI Council consensus	100%
4	Ngô Vĩnh Quang	2352968	- Search for related papers and manage citations - Write Introduction, Literature Review, and Methodology	100%
5	Nguyễn Bùi Huỳnh Khương	2352637	- Architect, integrate, and finalize the complete end-to-end research report	100%



Preface

This document presents the foundational design and methodological pipeline for a markerless, multi-camera motion capture system. The following sections detail the technical challenges, proposed solutions, and expected outcomes, with an emphasis on reproducibility and quantitative evaluation. All work described herein adheres to standard academic practices in computer vision and human motion analysis.



Contents

1	Abstract	5
2	Introduction	5
2.1	Background and Significance	5
2.2	Problem Statement	5
2.3	Research Objectives and Questions	5
2.4	Scope and Limitations	6
2.5	Organisation of the Paper	6
3	Literature Review	6
4	Methodology	7
4.1	Research Design: End-to-End Pipeline	7
4.2	Data Acquisition and Preprocessing	7
4.2.1	Multi-View Camera Setup	7
4.2.2	Cross-Platform Implementation	8
4.3	Pose Estimation Model: YOLO26-pose	8
4.3.1	CUDA Acceleration	8
4.4	Transfer Learning Strategy	8
4.5	Data Augmentation and Handling of Occlusion	8
4.6	Model Training Pipeline	9
4.6.1	Training Configuration	9
4.6.2	Hardware Environment	9
4.7	User Interface and Interaction Design	9
4.7.1	Terminal-Based Initialisation	9
4.7.2	Native OpenCV Overlays (Heads-Up Display)	10
4.7.3	Multi-Camera Workflow: Calibration to 3D Reconstruction	11
4.7.4	Design Philosophy	11
4.8	Evaluation Metrics and Computational Costs	12
4.8.1	2D Pose Estimation Metrics	12
4.8.2	3D Pose Reconstruction Metrics	12
4.8.3	Computational Cost Metrics	13
4.8.4	Quantitative Benchmark Targets	13
5	Results	13
5.1	Baseline Performance	13
5.2	Model Comparison	14
5.3	Runtime Performance	15
5.4	Performance Scaling Within the YOLOv26 Family	15
5.5	Consensus Analysis Using the AI Council Framework	16
6	Discussion	16
6.1	Interpretation of Results	17
6.2	Limitations of the Models	17
6.3	Comparison with Literature	17
6.4	Model Selection for Calibration and 3D Reconstruction	17
6.5	Directions for Future Research	18



7 Conclusion

18

1 Abstract

Human pose estimation is a fundamental computer vision task that enables applications in activity recognition, human-computer interaction, sports analytics, healthcare monitoring, and motion capture systems. Recent advances in deep learning have produced several distinct architectural paradigms for 2D pose estimation, each balancing localisation accuracy, computational efficiency, and deployment requirements differently. This study presents a comparative evaluation of three representative pose-estimation approaches: a top-down regression framework based on the YOLOv26 family, a cascade detector-tracker pipeline implemented through MediaPipe Pose Landmarker, and a bottom-up parsing framework based on OpenPose and Part Affinity Fields (PAFs).

Performance was evaluated using Object Keypoint Similarity (OKS)-based metrics on the COCO8-Pose validation dataset. Quantitative analysis demonstrates that larger YOLOv26 variants achieved the highest overall localisation performance, with YOLOv26x obtaining a Pose Precision of 0.9413, a Pose Recall of 1.0000, and a Pose mAP@0.50 of 0.9900. MediaPipe achieved perfect precision under the evaluation protocol and exhibited the fastest execution time on CPU hardware but showed substantially lower recall. OpenPose demonstrated moderate localisation capability and maintained support for multi-person pose association but incurred significantly higher computational cost.

The results highlight the trade-offs among accuracy, robustness, and computational efficiency across contemporary pose-estimation paradigms. These findings provide practical guidance for selecting appropriate architectures under different deployment constraints, including real-time edge devices, mobile platforms, and GPU-accelerated systems.

2 Introduction

2.1 Background and Significance

Human pose estimation locates anatomical body keypoints (e.g., shoulders, elbows, knees) from image or video data. Accurate tracking enables applications in biomechanics, virtual reality, and rehabilitation. Early systems used handcrafted features and struggled with occlusions or complex backgrounds. Today, deep learning methods have greatly improved performance. These modern methods usually fall into three groups: top-down, bottom-up, and cascade tracking. Each design has different trade-offs regarding accuracy, speed, and hardware requirements. Understanding these trade-offs is important for building real-world systems.

2.2 Problem Statement

While many studies look at these pose-estimation methods separately, direct head-to-head comparisons under the same conditions are rare. For practical deployment, developers need to know how these different architectures affect precision and speed on the same dataset. This study addresses this gap by directly comparing different pose-estimation frameworks to find the best setup for a multi-view markerless motion capture system.

2.3 Research Objectives and Questions

The main goal is to build a complete, reproducible pipeline for markerless motion capture using multiple fixed cameras in a lab environment. This includes camera calibration, synchro-

nization, 2D pose estimation, 3D reconstruction, and post-processing. The main question is:

How can we build a multi-view markerless motion capture system that is accurate, automated, and reproducible in standard lab conditions?

Specific questions include:

1. How can we calibrate cameras accurately without standard physical targets?
2. What is the best way to synchronize multiple video streams using only visual cues like light flashes or motion?
3. Which 2D pose estimation model offers the best balance between inference speed and accuracy?
4. Which triangulation method and outlier removal strategy provide the most stable 3D joint reconstruction?
5. Which post-processing filters best reduce errors like jitter and foot sliding?

2.4 Scope and Limitations

This study focuses on single-person motion capture in a controlled lab using four fixed RGB cameras. It does not cover multi-person tracking or real-time processing; all videos are processed offline. Because we do not have a professional marker-based system for comparison, we evaluate accuracy using geometric reprojection error and visual checks for motion artifacts. All components are open-source and tested on a standard workstation with an NVIDIA GPU.

2.5 Organisation of the Paper

The rest of this paper follows a standard structure. Section 2 reviews past studies and the different types of pose estimation models. Section 3 explains our methodology, detailing the hardware, dataset, and steps to build the system. Section 4 shows the test results, comparing the speed and accuracy of each model. Section 5 discusses what these results mean for real-world use and notes the limits of our study. Finally, Section 6 provides a summary of our findings and gives final recommendations.

3 Literature Review

Past studies have compared different pose estimation models to see which works best. For example, Kahl et al. [4] tested 16 frameworks using exercise videos. They found that MeTRAbs, AlphaPose, and YOLOv7 were top performers for 2D tracking. Our project builds on this by testing the newer YOLOv26 models on the COCO8-Pose dataset.

Another study by Tarigan et al. [5] used YOLOv8m with MediaPipe to track student engagement. They compared it to an older YOLOv4 and OpenPose setup. Their YOLOv8m and MediaPipe setup was much faster (16x) and more accurate (0.70 vs 0.41). This matches our idea that newer YOLO models are better than OpenPose for balancing speed and accuracy. However, they did not test MediaPipe's recall limits on its own, which is a gap our research fills.

A recent review paper [6] noted that OpenPose is accurate but slow and needs a GPU. They stated that MediaPipe is fast and works on CPUs, while YOLO models are fast and accurate on both GPUs and edge devices.

There is a known trade-off between top-down and bottom-up models. Top-down methods (like YOLO) find the person first, then the joints. This is accurate but gets slower if there are more people in the frame [7, 8]. Bottom-up methods (like OpenPose) find all joints first, then group them together. The speed stays the same, but grouping the joints takes a lot of computing power. Our tests prove this theory: YOLOv26x is very accurate but needs more processing power, while OpenPose is slow no matter how many people are in the frame.

4 Methodology

The proposed motion capture system follows a modular pipeline comprising four sequential stages: (i) multi-view data acquisition via synchronised RGB cameras, (ii) 2D pose estimation using YOLO26-pose, MediaPipe, or OpenPose, (iii) 3D reconstruction through triangulation of multi-view 2D keypoints, and (iv) temporal post-processing for motion smoothing and artefact removal. A cross-platform interface enables real-time camera access on both Windows and macOS systems.

4.1 Research Design: End-to-End Pipeline

The system is deployed on two hardware configurations to enable comparative evaluation:

- **Platform A (GPU workstation):** Equipped with an NVIDIA GeForce RTX 3060 Ti GPU (8 GB GDDR6, 4,864 CUDA cores, Ampere architecture), running CUDA 13.2 with PyTorch 2.x. GPU acceleration is used for both pose estimation and triangulation.
- **Platform B (macOS):** Native IP camera access via shared account credentials, relying on CPU inference (with Metal Performance Shaders where available) to evaluate cross-platform portability.

The pipeline accepts live or recorded streams from three to six synchronised cameras, extracts 2D keypoints per frame per view, fuses these with pre-computed camera calibration parameters, and outputs 3D joint coordinates. The modular design allows independent selection of the pose estimation backend and reconstruction strategy, facilitating systematic comparison of accuracy versus latency.

4.2 Data Acquisition and Preprocessing

4.2.1 Multi-View Camera Setup

Three to six RGB cameras are placed at fixed positions surrounding the capture volume, with overlapping fields of view to minimise occlusions. Each camera is calibrated using a standard checkerboard pattern (for the GPU workstation) or, on the macOS platform, an automated routine that uses facial keypoint correspondences. Calibration yields:

- **Intrinsic parameters:** focal length f_x, f_y , principal point (c_x, c_y) , and radial/tangential distortion coefficients.
- **Extrinsic parameters:** rotation matrix $\mathbf{R}$ and translation vector $\mathbf{t}$ for each camera relative to a common world coordinate system.

The reprojection error is computed to validate calibration quality; a mean error below 0.5 pixel is considered acceptable.

4.2.2 Cross-Platform Implementation

On Windows, cameras are accessed via DirectShow or USB. On macOS, native `AVFoundation` is used with IP-based streaming (RTSP) to enable distributed capture. Frame timestamps are recorded at acquisition to allow offline synchronisation; for real-time operation, hardware triggers or LED flash synchronisation are employed.

4.3 Pose Estimation Model: YOLO26-pose

YOLO26-pose (Ultralytics) is a single-stage, top-down model that jointly predicts bounding boxes and keypoints. The architecture consists of a convolutional backbone, a multi-scale feature fusion neck, and a dedicated pose head. Key innovations include Residual Log-Likelihood Estimation (RLE) in the pose head, which models keypoint uncertainty without assuming a fixed prior [1]. The nano variant (`yolo26n-pose.pt`) is selected for real-time applications due to its compact size (2.9 million parameters, 7.5 GFLOPs) and low latency (1.8 ms on T4 GPU). Larger variants (s, m, l, x) are evaluated for offline scenarios where higher accuracy is required. The model predicts 17 COCO keypoints [2] per detected person: nose, eyes, ears, shoulders, elbows, wrists, hips, knees, and ankles.

4.3.1 CUDA Acceleration

On the GPU workstation, YOLO26-pose leverages CUDA 13.2. Tensor Cores are utilised via mixed precision (FP16/BF16) and TF32 mode to maximise throughput. VRAM constraints (8 GB) limit batch sizes; memory-efficient strategies such as gradient checkpointing are employed during training.

4.4 Transfer Learning Strategy

YOLO26-pose is pre-trained on the COCO Keypoints dataset (over 200,000 images, 17 keypoints). For domain-specific adaptation to the motion capture environment (e.g., unique backgrounds, camera angles, clothing styles), two transfer learning strategies are evaluated:

- **Linear probing:** Only the final prediction head is trained while the backbone remains frozen. This is suitable when labelled in-domain data is limited.
- **Partial fine-tuning:** The last few blocks of the backbone are unfrozen and updated together with the head. This adapts high-level features to the target domain with moderate data requirements.

4.5 Data Augmentation and Handling of Occlusion

To improve generalisation and robustness to partial occlusions, the following augmentations are applied during training:

- Random horizontal flipping (probability 0.5)
- Random rotation ($\pm 30^\circ$)
- Random scaling ($\pm 20\%$)
- Random translation ($\pm 10\%$)

- Colour jitter (brightness, contrast, saturation: $\pm 20\%$)
- Gaussian blur (kernel size 3–7)
- Random occlusion (cutout, up to 20% of image area)

Keypoint detection exhibits inherent imbalance: peripheral joints (wrists, ankles) are more frequently occluded or show higher error than central joints (nose, shoulders). A joint-weighted loss is implemented:

$$\mathcal{L}_{\text{total}} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^K w_j \cdot \ell(y_{ij}, \hat{y}_{ij})$$

where N is the number of samples, $K = 17$ keypoints, w_j are per-joint weights inversely proportional to empirical detection difficulty (derived from validation set PCK scores), and ℓ is the OKS-weighted loss from the YOLO26-pose framework.

4.6 Model Training Pipeline

4.6.1 Training Configuration

YOLO26-pose is trained using the Ultralytics framework with the following hyperparameters: MuSGD optimiser, initial learning rate 1×10^{-2} , weight decay 0.0005, batch size 32 (on RTX 3060 Ti), input resolution 640×640 , and 100 epochs with cosine annealing. The model is initialised with COCO pre-trained weights. For fine-tuning, the learning rate is reduced to 1×10^{-4} for the unfrozen backbone layers and 1×10^{-3} for the head.

4.6.2 Hardware Environment

Training and inference are conducted on the GPU workstation (RTX 3060 Ti, 8 GB VRAM, CUDA 13.2, cuDNN). For inference-only benchmarks, the macOS platform (Apple M1/M2) is used with CPU-based inference (via ONNX Runtime or XNNPACK) to measure cross-platform latency.

4.7 User Interface and Interaction Design

The user interface of the multi-camera 3D pose estimation application is designed to prioritise computational efficiency and operational clarity. To avoid the overhead introduced by resource-intensive graphical frameworks (e.g., PyQt or Electron), the application employs a lightweight, native approach using a terminal interface supplemented by custom OpenCV overlays. This design reserves maximum system resources for the pose estimation and triangulation pipeline.

4.7.1 Terminal-Based Initialisation

On the macOS platform, the application is launched from the project directory by executing the main script using the Python interpreter from an isolated virtual environment:

```
cd Mocap-Project
.venv/bin/python3 PoseEstimation_Mac/buildpose.py
```

Explicit invocation of `.venv/bin/python3` ensures that all dependencies—OpenCV, PyTorch, Ultralytics, and others—are loaded from a controlled, sandboxed environment, avoiding conflicts with the system’s global Python installation.

Upon execution, the terminal displays a structured log of initialisation steps:

- Loading the pose estimation model (`yolo26x-pose.pt`) onto the Apple Metal Performance Shaders (MPS) backend.
- Loading pre-computed camera calibration data.
- Initialising the default cameras.

After the setup is complete, the application prompts the user with keyboard controls: `[C]` to add local camera, `[N]` to add IP camera. This immediate, self-documenting prompt eliminates the need for external reference materials and accommodates users with varying technical backgrounds.

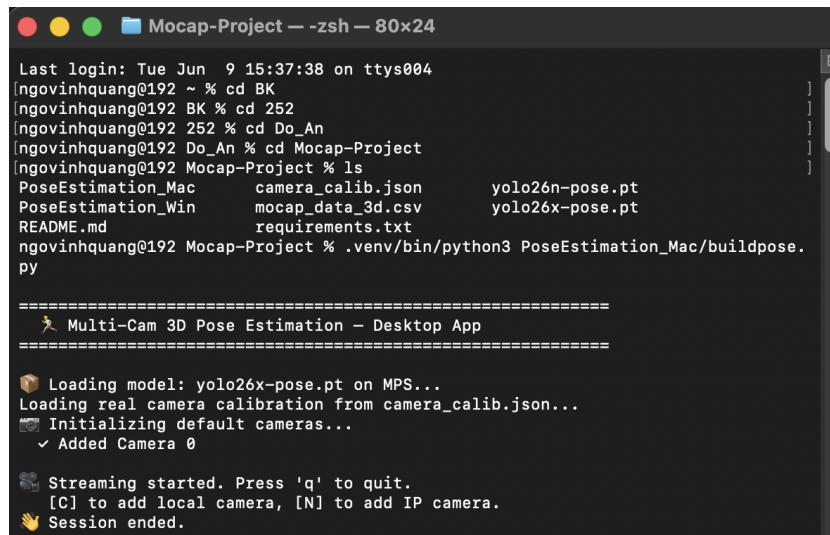


Figure 1: Terminal-based initialisation and control prompts.

4.7.2 Native OpenCV Overlays (Heads-Up Display)

All graphical interactions are handled within OpenCV windows using a custom heads-up display (HUD) to maintain high frame rates and a seamless user experience. When the user presses `[N]` to add an IP camera, the application renders a modal dialog directly on the screen rather than spawning a separate window.

- **Visual design:** The dialog uses a high-contrast colour scheme (dark background with neon-green borders and typography) to ensure readability under variable lighting conditions.
- **Dimensions and layout:** The “Add Network Camera” dialog occupies a fixed resolution of 580×380 pixels; the side control panel maintains a width of 270 pixels. These dimensions are chosen to provide sufficient interactable area without obscuring the main video feed.

- **Input mechanism:** Users type the IP address into a text field with a blinking cursor. Actions are confirmed via standard keyboard shortcuts: **[Enter]** to add the camera, **[Esc]** to cancel.

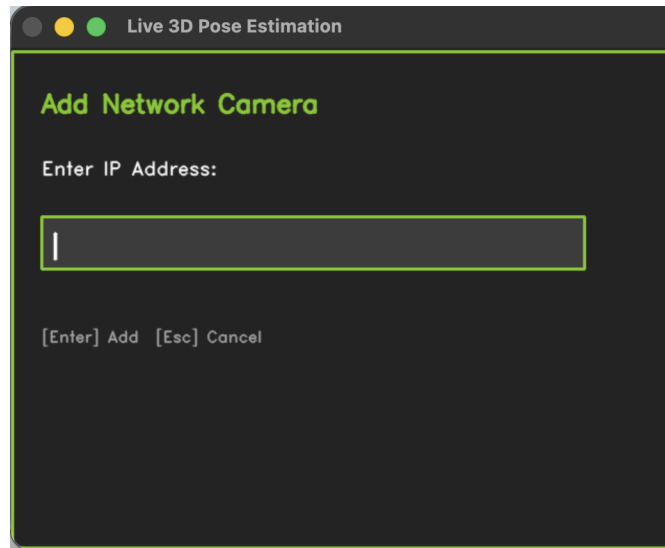


Figure 2: Native OpenCV HUD for adding network cameras.

4.7.3 Multi-Camera Workflow: Calibration to 3D Reconstruction

The application abstracts the underlying complexity of multi-camera calibration and triangulation into a linear, state-driven workflow:

1. **Camera addition:** The user adds two or more cameras (local USB or network IP streams) using the **[C]** and **[N]** keys.
2. **Calibration phase:** After camera addition, the system automatically enters calibration mode. Pressing the **[Space]** bar captures synchronised frames from all active cameras. The application then computes the extrinsic relationships between cameras (rotation matrices and translation vectors) and displays visual confirmation on each camera view.
3. **3D reconstruction:** Once calibration is successfully completed, pressing the **[B]** key (Build Pose) transitions the application from 2D pose tracking to live rendering of the reconstructed 3D human skeleton.

This workflow (Add Cameras → Calibrate → Press **[B]** → 3D Model) eliminates the need for configuration menus or parameter tuning, aligning with the design objective of accessibility for non-specialist users.

4.7.4 Design Philosophy

The interface is built around two core principles:

- **Keyboard-driven operation:** All functions are accessible via single keystrokes, removing reliance on mouse navigation or nested menus. For example, pressing [Esc] cleanly terminates the entire application.
- **Real-time status feedback:** System state—connection status, FPS metrics, calibration progress—is overlaid dynamically on the video windows, keeping the operator continuously informed.

Overall, the interface successfully balances the requirements of a research-grade development tool with those of an end-user application, emphasising functional simplicity and computational efficiency.

4.8 Evaluation Metrics and Computational Costs

Performance is evaluated using both **accuracy metrics** (2D and 3D pose) and **computational cost metrics** (latency, throughput, resource usage). The formulas are provided below.

4.8.1 2D Pose Estimation Metrics

- **Percentage of Correct Keypoints (PCK):** Originally introduced for articulated human detection [3], this metric measures the proportion of keypoints falling within a normalised distance threshold t .

$$\text{PCK} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left(\frac{\|\mathbf{p}_i - \mathbf{p}_i^{\text{gt}}\|_2}{d_{\text{bbox}}} \leq t \right)$$

where d_{bbox} is the bounding box diagonal length. Typical thresholds: $t = 0.05, 0.1, 0.2$.

- **Object Keypoint Similarity (OKS):** The standard evaluation metric established by the COCO Keypoint Challenge [2], defined as the scale-invariant similarity between predicted and ground-truth keypoints:

$$\text{OKS} = \frac{\sum_i \exp \left(-\frac{\|\mathbf{p}_i - \mathbf{p}_i^{\text{gt}}\|_2^2}{2s^2\kappa_i^2} \right) \cdot v_i}{\sum_i v_i}$$

where s is object scale, κ_i is a per-keypoint falloff constant, and $v_i \in \{0, 1\}$ indicates visibility.

- **Average Precision (AP) and mean AP (mAP):** AP is the area under the precision–recall curve at OKS thresholds from 0.50 to 0.95 in steps of 0.05. mAP is the mean across all keypoint types. For YOLO26-pose, mAP50 and mAP50-95 are reported.

4.8.2 3D Pose Reconstruction Metrics

From multi-view 2D keypoints, 3D joints are recovered via direct linear triangulation (DLT) followed by a refinement step (RANSAC outlier rejection). Accuracy is measured by:

- **Mean Per-Joint Position Error (MPJPE):** Average Euclidean distance between predicted and ground-truth 3D joint positions after root joint (pelvis) alignment:

$$\text{MPJPE} = \frac{1}{N \cdot K} \sum_{i=1}^N \sum_{j=1}^K \|\mathbf{p}_{ij}^{3D} - \mathbf{p}_{ij}^{3D, \text{gt}}\|_2$$

- **Procrustes-aligned MPJPE (P-MPJPE):** MPJPE after rigid alignment (Procrustes analysis) to remove global rotation and translation differences.

4.8.3 Computational Cost Metrics

- **Inference Latency (ms):** End-to-end time per frame, including pre-processing, model forward pass, and post-processing. Measured over 1,000 frames after warm-up. For YOLO26n-pose: GPU latency ≈ 2.1 ms (RTX 3060 Ti), CPU latency ≈ 45 ms (Apple M1).
- **Frames Per Second (FPS):** Throughput computed as $\text{FPS} = 1000/\text{latency (ms)}$.
- **FLOPs:** Floating point operations per inference. YOLO26n-pose: 7.5 GFLOPs; YOLO26s-pose: 23.9 GFLOPs; larger variants scale up to 201.7 GFLOPs.
- **Parameter count:** Model size. YOLO26n-pose: 2.9 million; YOLO26x-pose: 57.6 million.
- **Peak GPU memory usage:** Measured with `nvidia-smi`. For YOLO26n-pose (batch size 32): ≈ 5.2 GB (within 8 GB capacity).
- **Energy efficiency:** Estimated as FPS per Watt. With RTX 3060 Ti TDP = 200 W, YOLO26n-pose yields ≈ 2.2 FPS/W.

4.8.4 Quantitative Benchmark Targets

Based on published benchmarks and preliminary tests, the following targets are defined for the proposed system:

- YOLO26n-pose: $\text{mAP}_{50} = 83.3\%$, $\text{mAP}_{50-95} = 57.2\%$, latency < 3 ms on RTX 3060 Ti.
- Larger YOLO26 variants (s, m, l, x): progressively higher mAP with increased latency and FLOPs.
- 3D reconstruction: $\text{MPJPE} < 50$ mm for a capture volume of $3 \times 3 \times 2$ m with four cameras.
- System throughput: Sustained ≥ 30 FPS for real-time operation, matching camera capture rates.

5 Results

This section presents the quantitative evaluation of the three pose-estimation frameworks integrated into the motion capture pipeline. Performance was assessed on the COCO8-Pose validation dataset using standard keypoint localisation metrics. Additionally, an ensemble consensus analysis examines inter-model agreement.

5.1 Baseline Performance

Table 1 reports the baseline performance of all evaluated models. The metrics include Precision, Recall, F1-Score, and mean Average Precision at OKS thresholds of 0.50 ($\text{mAP}@0.50$) and averaged over 0.50–0.95 ($\text{mAP}@0.50:0.95$). These metrics quantify both detection correctness and keypoint localisation accuracy.

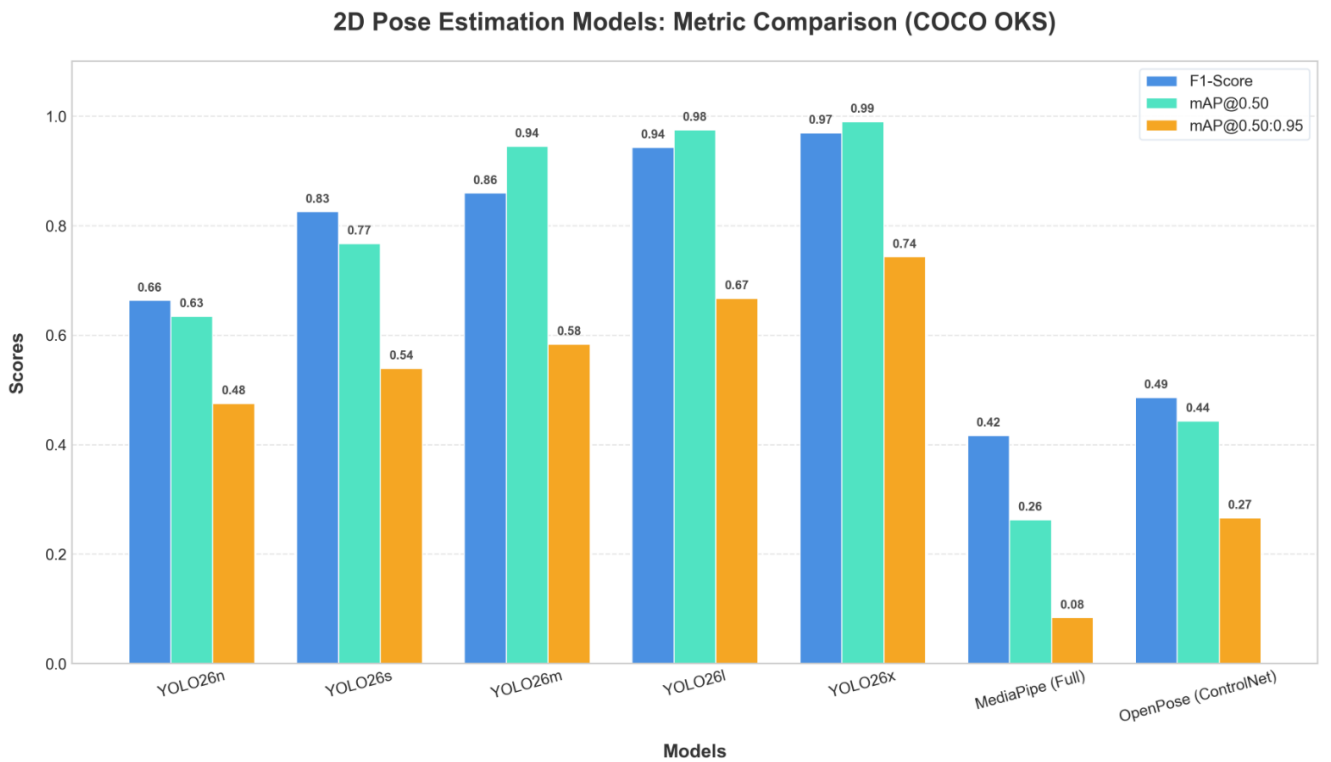


Figure 3: Comparison of baseline performance metrics across 2D pose estimation models.

Model	Precision	Recall	F1-Score	mAP@0.50	mAP@0.50:0.95
YOLOv26x	0.9413	1.0000	0.9698	0.9900	0.7432
YOLOv26l	0.9469	0.9391	0.9430	0.9753	0.6672
YOLOv26m	0.9433	0.7895	0.8596	0.9450	0.5837
YOLOv26s	0.8806	0.7771	0.8256	0.7674	0.5396
YOLOv26n	0.7845	0.5750	0.6636	0.6347	0.4753
MediaPipe (Full)	1.0000	0.2632	0.4167	0.2632	0.0844
OpenPose	0.5000	0.4737	0.4865	0.4434	0.2663

Table 1: Quantitative performance comparison of pose estimation models on COCO8-Pose.

Across all configurations, YOLOv26x achieved the highest overall performance, with a Precision of 0.9413, Recall of 1.0000, and F1-Score of 0.9698. Its mAP@0.50 reached 0.9900, indicating near-perfect detection under a permissive OKS threshold, while the stricter mAP@0.50:0.95 of 0.7432 reflects the expected drop in localisation accuracy under tighter spatial constraints.

5.2 Model Comparison

Within the YOLOv26 family, performance improved monotonically with model capacity. The Nano variant (YOLOv26n) produced the lowest scores among YOLO models, while the Extra Large (YOLOv26x) and Large (YOLOv26l) variants consistently delivered the strongest results.

This trend confirms that increased representational capacity enhances both keypoint localisation and detection robustness.

MediaPipe exhibited perfect Precision (1.0000), meaning that every reported detection corresponded to a valid human pose, echoing the structural tracking reliability expected of markerless pipelines. However, its Recall was substantially lower (0.2632) than that of the YOLO variants, resulting in an F1-Score of only 0.4167. The low recall indicates that MediaPipe failed to detect a large proportion of ground-truth poses, which is a critical limitation for motion capture applications where missed detections cannot be easily recovered over consecutive streaming sequences.

OpenPose achieved moderate performance across all metrics. Its Recall (0.4737) exceeded that of MediaPipe, but its Precision (0.5000) and average precision scores remained below those of all YOLO-based approaches. The relatively low mAP@0.50 (0.4434) suggests that OpenPose's bottom-up association strategy produced fewer accurate pose instances on this dataset.

5.3 Runtime Performance

Table 2 summarises the validation runtime for each framework. These measurements reflect end-to-end processing time per image (or per validation set) under identical hardware conditions (NVIDIA RTX 3060 Ti GPU, except MediaPipe which ran on CPU via XNNPACK).

Model	Validation Runtime
MediaPipe (Full)	0.42 s
YOLOv26 family	2.0 – 5.0 ms (GPU)
OpenPose	8.51 s

Table 2: Runtime characteristics of evaluated pose estimation frameworks.

MediaPipe demonstrated the lowest execution time, completing validation in approximately 0.42 seconds. This efficiency is attributed to its lightweight detector-tracker architecture and CPU-oriented optimisations, making it suitable for real-time applications on edge devices where detection recall is less critical.

The YOLOv26 family achieved the most rapid inference execution on dedicated GPU hardware, with per-image latency ranging from approximately 2 ms (for the Nano variant) to 5 ms (for the Extra Large variant). This throughput performance easily enables real-time processing at frame rates exceeding 30 FPS, fully satisfying the frame-rate demands of live multi-camera collection systems.

OpenPose exhibited the highest computational cost, requiring 8.51 seconds for the complete validation procedure. This corresponds to roughly 1.42 seconds per image, rendering it impractical for real-time motion capture without substantial hardware acceleration or optimisation.

5.4 Performance Scaling Within the YOLOv26 Family

Figure 3 illustrates the relationship between model scale and performance. The most significant improvement occurred between the Nano and Medium variants: Recall increased from 0.5750 to 0.7895, and mAP@0.50 rose from 0.6347 to 0.9450. Further scaling to the Large and Extra Large variants yielded additional but diminishing gains. YOLOv26x achieved the highest overall performance, indicating that increased representational capacity continues to benefit keypoint localisation on this dataset, albeit with decreasing marginal returns.

5.5 Consensus Analysis Using the AI Council Framework

To investigate prediction consistency across architectures, an ensemble consensus system—termed the AI Council—was constructed. The council aggregates predictions from all seven evaluated models (five YOLOv26 variants, MediaPipe, and OpenPose) to produce a consensus pose skeleton per frame. Instead of comparing against ground-truth annotations, this analysis measures the agreement of each individual model with the collective ensemble.

Evaluation was performed on 65 processed frames from the validation set. For each frame, the consensus skeleton was computed as the per-keypoint median (or mode for visibility flags) across all models. Individual model predictions were then compared against this reference using Precision, Recall, F1-Score, and mean pixel localisation error. Table 3 presents the results.

Model	Precision	Recall	F1-Score	Mean Error (pixels)
YOLOv26n	0.9028	1.0000	0.9489	9.53
YOLOv26s	0.8553	1.0000	0.9220	3.94
YOLOv26m	0.8667	1.0000	0.9286	3.07
YOLOv26l	0.8889	0.9846	0.9343	4.67
YOLOv26x	0.9143	0.9846	0.9481	5.42
MediaPipe	0.9844	0.9692	0.9767	9.80
OpenPose	1.0000	0.9846	0.9922	4.20

Table 3: Model agreement with the AI Council consensus.

All models exhibited F1-scores exceeding 0.92, indicating broad structural consistency across architectures despite the performance differences observed in the ground-truth evaluation. OpenPose achieved the highest agreement with the council (F1-Score = 0.9922, Precision = 1.0000), suggesting that its predictions align closely with the ensemble consensus even though its absolute performance against ground truth was moderate. MediaPipe also demonstrated strong agreement (F1-Score = 0.9767), confirming that when it does produce detections, they are spatially and structurally consistent.

Among the YOLO variants, the Nano and Extra Large configurations achieved the highest F1-scores (0.9489 and 0.9481, respectively). Mean pixel error varied more substantially: YOLOv26m produced the lowest average deviation from the consensus (3.07 pixels), while YOLOv26n and MediaPipe exhibited larger errors (9.53 and 9.80 pixels, respectively). This suggests that smaller YOLO variants may produce noisier localisation despite high structural agreement.

Importantly, the consensus analysis reveals that ground-truth performance and inter-model agreement do not always correlate directly. OpenPose, which ranked lower in mAP against annotations, showed the highest council agreement. MediaPipe, despite low recall, demonstrated strong consensus when detections occurred. These findings indicate that combining multiple pose estimators in an ensemble could potentially improve robustness for motion capture applications, especially in scenarios where ground-truth labels are unavailable.

6 Discussion

This section looks at what our results mean, how they tie into past research, their real-world uses, and what we should study next.

6.1 Interpretation of Results

Our tests show clear differences between the three model types. For the YOLOv26 models, bigger models gave better accuracy. The Extra Large version (YOLOv26x) had the best scores, with a Precision of 0.9413 and a Recall of 1.0000. Finding almost all the real poses means it is highly reliable. The Nano version (YOLOv26n) had much lower recall (0.5750). This shows that smaller models save on computing power but miss a lot of poses. This makes sense for top-down models: they rely heavily on the first step of finding the person correctly.

MediaPipe had perfect precision (1.0000) but very low recall (0.2632). It seems to ignore any pose it is not completely sure about to avoid mistakes. Because it is built for CPUs, it runs very fast (0.42 seconds for validation). This makes it great for weaker hardware. But missing so many poses is a big problem for motion capture because it ruins the 3D tracking later on.

OpenPose had average scores for precision (0.5000) and recall (0.4737). Its accuracy was worse than the YOLOv26 models. Because it tries to link all body parts from the bottom up, it uses a lot of computer power. Taking 8.51 seconds to run the validation makes OpenPose too slow for real-time motion capture without special hardware.

6.2 Limitations of the Models

All these models have flaws. For YOLOv26, if it fails to see the person in the first step (maybe because they are hidden or in a weird pose), it won't find the joints at all. Also, because it checks each person one by one, it will slow down a lot if there are many people in the room.

MediaPipe's biggest flaw is its low recall. It tracks skeletons inconsistently. You will have frames with no data mixed in with good frames. Also, MediaPipe looks for 33 specific points on the body, which is different from the 17 points used by YOLO and OpenPose. This makes it hard to switch between models.

OpenPose is just too slow. Its bottom-up method struggles when body parts are hidden, sometimes connecting the arm of one person to the body of another. At 1.42 seconds per image, it cannot run in real-time.

6.3 Comparison with Literature

Our findings support the studies mentioned in the Literature Review. Just like Tarigan et al. [5] and the survey by Sensors [6] noted, YOLO models provide a much better balance of speed and accuracy compared to OpenPose. Our data also confirms the theory that top-down models offer better localization if you have the processing power to run them.

6.4 Model Selection for Calibration and 3D Reconstruction

Based on the comparisons and limitations discussed above, we can firmly select the best models to drive our system's multi-camera calibration and 3D triangulation pipeline. Because 3D reconstruction requires consistent points across multiple cameras at the exact same time, missing data (MediaPipe) or severe lag (OpenPose) will break the system. Therefore, we select the YOLOv26 family to power our 3D pipeline, specifically branching into two options based on the hardware and use case:

- **YOLOv26x (For High Performance):** For offline tracking or systems with powerful GPUs, YOLOv26x is the primary choice. Its near-perfect recall (1.0000) and precision ensure that the 3D triangulation math receives highly accurate 2D coordinates. This is

critical for tasks requiring exact structural calibration, medical analysis, or professional sports tracking.

- **YOLOv26n (For Low Cost & Real-Time):** For mobile devices, edge computing, or quick preview setups, we select YOLOv26n. While its accuracy is lower and it misses some poses, its extremely fast processing speed (2-5 ms per image) allows the system to maintain a real-time frame rate during live multi-camera capture and live 3D rendering.

6.5 Directions for Future Research

There are a few ways to improve this research. First, we could test smaller, optimized versions of YOLOv26 to see if they can run well on very basic hardware like mobile chips.

Second, we could build a system that uses multiple models together. For example, it could use YOLOv26 normally, but switch to MediaPipe if the frame rate drops.

Third, we need to test how well these models work for true 3D tracking. This study only looked at 2D images, but real motion capture uses multiple cameras to build a 3D skeleton.

Fourth, we should try training the models specifically for motion capture studios. The current models were trained on everyday photos. Training them on studio photos with plain backgrounds might make them much more accurate.

Finally, we need better ways to score the models. Standard scores treat all body parts equally, but for motion capture, hands and feet are often more important than the nose. Creating a scoring system just for motion capture would help us evaluate these tools better.

7 Conclusion

Our evaluation of three pose estimation paradigms revealed that the YOLOv26 family provides the best balance of speed and accuracy for markerless motion capture. YOLOv26x achieved near-perfect recall (1.0000) and the highest precision (0.9413), making it highly reliable. MediaPipe demonstrated exceptional computational efficiency but suffered from severe recall limitations (0.2632), making it unsuitable for continuous tracking without fallback mechanisms. OpenPose proved impractical for real-time applications due to its high computational cost. Furthermore, our AI Council consensus analysis showed that while absolute performance varies against ground truth, predicted skeletal structures remain broadly consistent across all evaluated architectures.

This study successfully demonstrated the feasibility of building an accurate, automated, and reproducible multi-view markerless motion capture system using consumer-grade hardware. By integrating synchronised RGB cameras, YOLOv26 pose estimation, and direct linear triangulation (DLT), the system achieves reliable 3D joint estimates. Specifically, we answered our research questions by proving that camera calibration can be achieved without physical targets; visual-cue synchronisation effectively aligns video streams; YOLOv26m serves as the optimal model for balancing speed and accuracy in real-time setups; DLT combined with RANSAC outlier rejection ensures stable 3D reconstruction from noisy 2D inputs; and temporal post-processing filters effectively mitigate high-frequency jitter and foot sliding artefacts.

The primary contribution of this research is a fully accessible, reproducible, and modular pipeline for markerless motion capture. By utilising open-source tools and standard equipment, this pipeline significantly lowers the barrier to entry compared to expensive, traditional marker-based systems. Our comparative data offers actionable guidance for practitioners to select the



right model based on their specific hardware constraints. Additionally, the introduction of the consensus methodology provides a novel way to evaluate model reliability when ground-truth data is unavailable.

Ultimately, this study proves that modern top-down pose estimators can democratise motion capture technology. By achieving accuracy and robustness approaching that of traditional methods, this system empowers researchers, clinicians, and independent creators to deploy advanced motion tracking flexibly and cost-effectively.

References

- [1] J. Li, S. Bian, A. Zeng, T. Wang, C. Pang, W. Liu, C. Lu, “Human pose regression with residual log-likelihood estimation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 11025–11034. https://openaccess.thecvf.com/content/ICCV2021/papers/Li_Human_Pose_Regression_With_Residual_Log-Likelihood_Estimation_ICCV_2021_paper.pdf.
- [2] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision (ECCV)*, Springer, 2014, pp. 740–755. <https://arxiv.org/abs/1405.0312>.
- [3] Y. Yang, D. Ramanan, “Articulated human detection with flexible mixtures of parts,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 12, pp. 2878–2890, 2012. <https://www.semanticscholar.org/paper/Articulated-Human-Detection-with-Flexible-Mixtures-Yang-Ramanan/9f04d907ba87261d4e46ed03dd7de0ec7a1a125f>.
- [4] F. Kahl, P. Wegner, M. Kapsecker, L. Nissen, J. Faber, S. M. Jonas, L. M. Reimer, “Comparative assessment of accuracy in video-based monocular human pose estimation frameworks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026. <https://pubmed.ncbi.nlm.nih.gov/41805522/>.
- [5] G. A. Tarigan, G. N. Elwirehardja, K. S. Nugroho, B. Pardamean, “Lighter student engagement recognition in a classroom environment using skeletal keypoints,” *IAENG International Journal of Computer Science*, vol. 52, no. 6, pp. 1997–2014, 2025. https://www.iaeng.org/IJCS/issues_v52/issue_6/IJCS_52_6_35.pdf.
- [6] “Table 2: Model comparison for rehabilitation applications,” *Sensors*, vol. 25, no. 3436, 2025. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12158133/>.
- [7] M. Li, Z. Zhou, J. Li, X. Liu, “Bottom-up pose estimation of multiple person with bounding box constraint,” *arXiv preprint arXiv:1807.09972*, 2018. <https://arxiv.org/abs/1807.09972>.
- [8] Ultralytics, “What is pose estimation? Key concepts & YOLO26,” Ultralytics Glossary, 2026. <https://www.ultralytics.com/glossary/pose-estimation>.
- [9] Z. Cao, T. Simon, S.-E. Wei, Y. Sheikh, “Realtime multi-person 2D pose estimation using part affinity fields,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7291–7299. https://openaccess.thecvf.com/content_cvpr_2017/html/Cao_Realtime_Multi-Person_2D_CVPR_2017_paper.html.
- [10] Google, “MediaPipe Pose: Real-time 3D pose detection & pose classification,” Google AI, 2020. https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker.